```
function top()
  if pairs.isEmpty() then error()
                 #1
  p ←
pairs.removeLast()
         #2
  if pairs.isEmpty()
then                                                    #3
    return p.element
  else
   (element, priority) ←
pairs[0]                                              #4
    pairs[0] ←
p
#5
    pushDown(pairs,
0)                                                     #6
    return element
```

#1 We need to check that the heap is not empty. Otherwise, we throw an error (or return null)

#2 Remove the last element in the `pairs` array, and store it in a temporary

#3 Now check again if there are more elements left: if not, just return `p`

#4 Otherwise we store the top pair (the first in the `pairs` array) in temporary variables…

#5 …and overwrite the first pair in the array with the previously saved `p`

#6 The last element was a leaf, so it likely had a low priority. Now that it sits at the root, it might violate the heap properties, so we need it to move it towards the leaves, till both its children have lower priority than it.